

HANDS-ON WORKSHEET

AI-Assisted Debugging: Stack Trace vs. AI

Curiosity Trigger: “Can AI explain a bug better than a stack trace can?”

Learning goal: Students will distinguish between runtime evidence and AI-generated explanations, identify logical bugs, write strong debugging prompts, and verify AI suggestions through testing.

Name	Reg. No.	Batch	Date
_L.SOWMYA_____	_2503A51670_____	__66_____	__3Sep2026_____
—	—	—	—

Activity 1 – PREDICT

Before running the code or asking AI, make a prediction. The program below contains a logical bug. It runs without producing a Python error, but the result can be wrong.

```
def calculate_discount(price, is_member):
    discount = 0

    if is_member:
        discount = 10
    else:
        discount = 0

    final_price = price discount / 100
    return final_price

print(calculate_discount(1000, True))
```

Predict before testing:

- What do you think the output should be for price = 1000 and is_member = True?

Answer: 900

- Do you expect a stack trace? Why or why not?

Answer: Yes, you will get a stack trace. The line is invalid syntax since there's no operator between price and discount

- What do you think is causing the problem?

Answer: The problem is caused by a missing operator

Activity 2 – EXPERIMENT: Run and Observe

Run the program in VS Code/Python. Record the actual output. Notice whether a stack trace appears.

Test	Student Record
Input	price = 1000, is_member = True
Actual output	_____error_____

Stack trace?	Yes If yes, paste or summarize it: <code>final_price = price discount / 100</code> SyntaxError: invalid syntax_____
--------------	--

Activity 3 – Ask AI a WEAK Prompt

Use this prompt exactly:

“My code is not working, fix it.”

Record what AI suggests:

AI diagnosis: `final_price = price discount / 100`

AI fix: `final_price = price - (price * discount / 100)`

Do you trust the answer immediately? No

Why? Because even though the fix looks logically correct, I would still need to run the code to confirm it produces the expected output without errors.

Activity 4 – Ask AI a STRONG Prompt

Now provide context, code, input, actual output, expected output, and a specific instruction.

Here is my `calculate_discount()` code.

Input: `price = 1000, is_member = True`
 Actual output: [paste your output]
 Expected: A 10% member discount should be applied.
 Error/stack trace: No error message was produced.

Find the logical bug.
 Explain the ROOT CAUSE first.
 Then provide the corrected code.
 Finally, explain how I can test that the fix is correct.

Record AI's response:

Root cause: `final_price = price discount / 100`

Suggested fix: `final_price = price*discount / 100`

Suggested test: Run the function with `price = 1000` and `is_member = True`; the output should be **900**, and with `is_member = False` the output should remain **1000**, confirming the fix works.

Activity 5 – Verify AI Instead of Trusting AI

Modify the code only after understanding the explanation. Then test both member and non-member cases.

Test Case	Expected	Actual	Pass/Fail
-----------	----------	--------	-----------

1000, True	900	900	pass
1000, False	1000	1000	pass
500, True	450	450	pass

Activity 6 – Stack Trace Challenge

Run this code and study the stack trace:

```
def book_slot(doc_id):
    doctors = {
        "CARDIO-01": "Dr. A",
        "CARDIO-02": "Dr. B"
    }
    return "Booking confirmed for " + doctors[doc_id]

print(book_slot("CARDIO-07"))
```

Answer:

1. What is the exception type?

____ KeyError _____

2. Which line caused the error?

return "Booking confirmed for " + doctors[doc_id]

3. What does the stack trace tell you?

The stack trace shows the code tried to access a missing dictionary key "CARDIO-07" and pinpoints the exact line causing the error

4. What does it NOT tell you about the intended business logic?

It does not explain what the program should do when an invalid doctor ID is given, such as showing a friendly message or suggesting valid IDs

5. Write a strong AI prompt asking for the root cause and a safe fix.

Strong AI prompt: *"My doctor booking code fails with a KeyError for invalid IDs; explain the root cause and provide a safe fix that handles unknown IDs gracefully."*

Activity 7 – AI Can Be Wrong

Ask AI to diagnose the previous problem. Then deliberately challenge the response.

Use this verification prompt:

"Do not assume your first diagnosis is correct. Identify the evidence in the code that supports your diagnosis. Give me one test that could prove your diagnosis wrong."

What evidence did AI provide?

AI pointed to the dictionary lookup `doctors[doc_id]` as evidence, showing that "CARDIO-07" is not a valid key in the dictionary.

What test did AI suggest? AI suggested running the function with a valid key like "CARDIO-01" to confirm that the error only occurs with invalid keys.

Did your test confirm the diagnosis? Yes

What did you learn? The stack trace identifies where the error occurs, but only testing reveals whether the diagnosis is correct and whether the fix truly solves the problem.

Activity 8 – Create Your Own Debugging Prompt

Create a strong prompt for a logical bug in a small Python program. Your prompt must contain all six elements:

- Context / purpose of the program
- Relevant code
- Input
- Actual output or behavior
- Expected output or behavior
- Instruction: explain ROOT CAUSE first and propose a test

Your prompt:

```
def square(x):
```

```
    return x + x
```

```
print(square(5))
```

Input: 5

Actual output: 10

Expected output: 25

Instruction: Explain the ROOT CAUSE first, then propose a test to confirm the fix

Activity 9 – Predict → Experiment → Record Reflection

Stage	Your observation
Predict	What did you think would happen? I thought the function would return the square of the input, so <code>square(5)</code> should give 25
Experiment	What happened when you ran the code and asked AI? Running the code gave 10, and AI explained that the bug was using addition instead of multiplication.
Record	Where was AI correct or wrong? What evidence proved it? the evidence was the code line <code>return x + x</code> . Testing with <code>square(5)</code> after changing to <code>return x * x</code> confirmed the fix.

Key Takeaway

STACK TRACE = evidence about where/how execution failed

AI = a reasoning assistant that may explain possible causes

TESTING = the final check that determines whether the explanation and fix are correct

Student challenge: Never accept an AI-generated fix only because it sounds confident. Understand the root cause, test the proposed fix, and record the evidence.